

INFORME TÉCNICO MAESTRO

PaseCultural

Índice

Estado actual del sistema

1. RESUMEN EJECUTIVO
2. ARQUITECTURA ACTUAL
3. ROLES Y PERMISOS
4. MÓDULOS FUNCIONALES
5. FLUJO ACTUAL DE WHATSAPP
6. BACKEND Y API
7. BASE DE DATOS
8. SEGURIDAD Y CONSISTENCIA
9. INSTALACIÓN, PRUEBAS Y DESPLIEGUE
10. LIMITACIONES Y PENDIENTES REALES
11. GUÍA DE TRASPASO
12. HISTORIAL RESUMIDO

Estado actual del sistema

Documentación interna de entrega técnica. Describe el sistema tal como está construido hoy — qué es, cómo funciona, qué está implementado y qué queda pendiente — para que un desarrollador nuevo pueda entenderlo, mantenerlo y continuarlo sin necesitar la historia previa del proyecto. No es una bitácora de construcción ni un registro de commits: es el plano del sistema terminado. Basado exclusivamente en el código presente en el repositorio; donde el código y una afirmación anterior discrepaban, este documento sigue al código.

Fecha de actualización: 14/08/2026 · **Versión:** 2026-08-14

1. RESUMEN EJECUTIVO

Qué problema resuelve. PaseCultural es una plataforma de venta y administración de entradas para eventos culturales (teatro, música, centros culturales, productoras independientes). Cubre el ciclo completo: un organizador crea y publica un evento con sus funciones y tipos de entrada (por Web o conversando con un bot de WhatsApp), el público compra sin necesidad de crear una cuenta, cada entrada se emite como un ticket individual con QR cifrado, el ingreso se controla en la puerta con una app de "Scanner" operada por personal sin cuenta de usuario tradicional, y tanto el organizador como el equipo de PaseCultural (rol "Developer") administran todo desde paneles propios con datos reales.

Estado general del producto. Núcleo funcional completo y en uso: catálogo de eventos, checkout de compra, entradas con QR cifrado, control de acceso con garantías de concurrencia verificadas, recuperación de compra, administración de entradas con auditoría, cortesías, ventas, estadísticas reales para organizador y para el equipo interno, y un canal completo de creación de eventos por WhatsApp que usa el mismo motor que la Web. El hueco más grande del sistema, con diferencia, es que **no existe ningún cobro real** — comprar una entrada hoy es funcionalmente gratis del lado técnico, sin importar el precio configurado.

Módulos operativos (funcionando de punta a punta, backend + frontend, con datos reales): Autenticación, Organizaciones, Eventos (Web y WhatsApp), Funciones, Entradas y tipos de tickets, Ventas, QR y control de acceso, Scanners, Cortesías, Recuperación de compra, Emails, Panel Developer, Panel Organizer, Configuración.

Componentes pendientes: integración de pago real (Mercado Pago — la ausencia más importante), exportación real de entradas seleccionadas, página de Términos de Servicio dedicada, deduplicación de reintentos de Meta en WhatsApp, timeout de conversación de WhatsApp por inactividad, limpieza de un lote de datos de prueba que quedó en producción (ver §11).

2. ARQUITECTURA ACTUAL

Monorepo con dos aplicaciones independientes, sin capa intermedia: `backend` (API REST) y `frontend` (SPA), que se comunican por HTTP/JSON. No hay SSR, no hay BFF, no hay microservicios.

Frontend

React 19, Vite, React Router 7, Tailwind CSS 4. Autenticación vía Clerk (`@clerk/clerk-react` + `@clerk/localizations` en español). Mapas y autocompletado de direcciones vía Google Maps JavaScript API (`@googlemaps/js-api-loader`). `qrcode.react` (render de QR en pantalla), `qr-scanner` (lectura de QR por cámara, módulo Scanner), `jspdf` (PDF de entradas generado en el navegador), `lucide-react` (iconografía). Sin librería de estado global (Context de React + `useState/useEffect`), sin librería de formularios, sin librería de componentes UI de terceros (sistema de diseño propio sobre Tailwind), sin i18n propio (textos en español hardcodeados; sólo la UI de Clerk está localizada), sin SDK de analítica/monitoreo, sin infraestructura de testing.

Backend

Node.js con Express 5 (ESM puro, `type: module`). Prisma 6 como ORM sobre PostgreSQL. `@clerk/express` para autenticación de usuarios con cuenta. `jsonwebtoken` (JWT propio, sesiones de Scanner). `bcrypt` (hashing del módulo Scanner/verificación, no de contraseñas de usuario — Clerk las maneja). `resend` (email transaccional). `cloudinary` (imágenes). `pdfkit/qrcode` (PDF y QR generados del lado servidor). `multer` (upload de archivos). `pg` (driver Postgres). Sin TypeScript, sin linter configurado, sin framework de testing externo — usa el test runner nativo de Node (`node:test`).

PostgreSQL / Prisma

Base de datos gestionada enteramente por Prisma (schema versionado + migraciones SQL). 34 migraciones aplicadas, 22 modelos, 22 enums (detalle completo en §8). Alojada en Supabase — hay **dos proyectos Supabase separados**: uno de producción y uno de TEST, distinguidos internamente por project-ref y nunca por nombre en ningún log o documento (ver §9, `guardrail dbGuard.js`).

Clerk

Autenticación y gestión de sesión para cualquier persona con cuenta (comprador registrado, organizador, developer). El backend nunca valida contraseñas ni sesiones por su cuenta — delega 100% en el SDK (`clerkMiddleware()` en `app.js`, lee sus credenciales de variables de entorno por convención propia del SDK, nunca pasadas explícitamente en el código). El módulo Scanner es la única parte del sistema que **no** usa Clerk (sesión propia, ver §4/§9).

Render

Host del backend. Confirmado por código, no sólo por convención: el backend lee `process.env.RENDER_GIT_COMMIT`, una variable que Render inyecta automáticamente en cada deploy — su sola presencia como variable esperada en el código es evidencia de que el backend

corre en Render. No hay `render.yaml` versionado en el repositorio — la configuración del servicio (build command, variables de entorno, auto-deploy desde `main`) vive en el dashboard de Render, fuera de este repositorio.

Vercel

Host del frontend. `frontend/vercel.json` está versionado en el repo, con una única regla de rewrite (`/(.*)` → `/index.html`), necesaria para que las rutas de React Router funcionen en carga directa/refresh). El build (`vite build`) y el deploy se gestionan por la integración estándar de Vercel con el repositorio Git.

Cloudinary

Almacenamiento de imágenes (portadas de evento, logos de organización). Subida genérica vía `POST /api/media/upload` (5 MB máx, PNG/JPEG/WEBP), borrado por `publicId`.

Resend

Envío de los 3 emails transaccionales del sistema (ver §4, Emails). Cliente inicializado de forma perezosa (`getRequiredEnv`), nunca al importar el módulo.

WhatsApp Cloud API (Meta Graph API)

Canal alternativo de creación de eventos para organizadores, vía un bot conversacional. Webhook público (`GET/POST /api/whatsapp/webhook`) y envío de mensajes salientes vía Graph API. Detalle completo del flujo vigente en §5.

Relación entre componentes

```
Frontend (Vercel, SPA)
├─ Clerk (sesión, directo desde el navegador)
├─ Google Maps JS API (directo desde el navegador)
└─ HTTP/JSON → Backend API (Render)
    └─ PostgreSQL (Supabase, vía Prisma)
        └─ Clerk (verificación de sesión)
        └─ Cloudinary (imágenes)
        └─ Resend (emails)
        └─ Google Maps Geocoding API (best-effort)
        └─ Meta Graph API (WhatsApp)

Organizador (WhatsApp) <—mensaje—> Meta <—webhook—> Backend API (Render)
```

El motor de creación de eventos (`EventCreationEngine / EventServicePort`) es la única pieza de lógica de negocio compartida entre ambos canales de entrada (Web y WhatsApp) — cada canal sólo aporta su propia capa de interpretación de entrada/salida (formulario/chat vs. texto libre de WhatsApp) sobre el mismo motor y las mismas tablas.

3. ROLES Y PERMISOS

Enum `Role` en la base: `DEVELOPER`, `ORGANIZER`, `SCANNER`, `CUSTOMER`. Autorización siempre resuelta server-side (`requireRole`, nunca confiada desde el cliente); dentro del panel de organizador, además del rol, cada acción revalida que el recurso (evento, scanner, entrada, venta) pertenezca a la organización de quien hace el pedido.

Rol	Cómo se obtiene	Capacidades reales	Restricciones
Developer	Lista de emails hardcodeada en <code>auth.service.js</code> , asignada al sincronizar la cuenta (<code>POST /api/auth/sync</code>)	Aprobar/rechazar/suspender organizaciones; gestionar rol/estado de cualquier usuario; ver plataforma completa sin acotarse a una organización (eventos, ventas, entradas, scanners de todos los organizadores, sólo lectura); Dashboard con KPIs globales; panel de Herramientas de Desarrollo (reset/seed de base, envío de WhatsApp de prueba)	No puede cancelar una cortesía de una organización ajena (limitación heredada, no ampliada a propósito)
Organizer	Automático al crear una organización (<code>POST /api/organizations</code> , promueve de <code>CUSTOMER</code>)	CRUD completo de sus propios eventos, funciones, tipos de entrada, links; emitir cortesías; administrar entradas vendidas (cancelar/rehabilitar/reactivar/marcar usada/eliminar); gestionar scanners de sus eventos; ver ventas y dashboard de su organización; vincular un número de WhatsApp a su organización	Siempre acotado a los eventos/recursos de su propia organización; sólo puede publicar eventos si la organización está <code>APPROVED</code> por un Developer
Scanner	Registro único por invitación del organizador (<code>EventScanner</code> , no requiere Clerk ni cuenta <code>User</code>)	Login recurrente por email + código de 6 dígitos; validar entradas en dos pasos (ver preview de la entrada, luego confirmar el ingreso); ver estadísticas de la función que tiene asignada	Sesión propia por JWT (<code>SCANNER_SESSION_SECRET</code> , 24 hs), revalidada <code>ACTIVE</code> en cada request — una desactivación del organizador corta el acceso al instante. El rol <code>SCANNER</code> del enum <code>Role / User</code> existe pero está en desuso: los scanners reales son filas <code>EventScanner</code> , independientes de Clerk
Customer	Rol por defecto de cualquier <code>User</code> sin organización propia	Comprar entradas (con o sin cuenta), ver "Mis entradas" si tiene cuenta, recuperar una compra sin cuenta por email+DNI+código	Sin acceso a ningún panel administrativo

4. MÓDULOS FUNCIONALES

Para cada módulo: propósito, estado actual, funcionamiento y limitaciones reales — verificado contra el código vigente.

Autenticación

Propósito: identificar quién es cada usuario y qué puede hacer. **Estado: completo.** Login/registro delegado 100% a los componentes prebuilt de Clerk; sincronización on-demand de la sesión de Clerk a una fila `User` propia (`POST /api/auth/sync` , sin webhook de Clerk); adopción automática de compras de invitado previas cuando alguien se registra con el mismo email que usó para comprar sin cuenta; asignación de rol `DEVELOPER` por lista hardcodeda de emails; promoción automática a `ORGANIZER` al crear una organización. **Limitaciones:** no hay webhook de Clerk — la sincronización depende de que el frontend llame a `/api/auth/sync` después de cada login; no hay flujo de "olvidé mi contraseña" propio (lo maneja Clerk).

Eventos

Propósito: que un organizador arme y publique un evento con sus funciones y entradas. **Estado: completo, con dos canales de creación (Web y WhatsApp) sobre el mismo motor.** CRUD completo (crear/listar/editar/eliminar/publicar/despublicar/cancelar/archivar/restaurar/duplicar); motor conversacional tipo "chat" (Web y WhatsApp) para crear eventos paso a paso, con posibilidad de retomar, editar cualquier sección desde una vista previa y guardar como borrador; wizard clásico de formulario para editar eventos ya existentes; enlaces del evento (redes/video, detección automática de plataforma); ubicación con Google Maps (Web) o compartida/manual (WhatsApp); listado público con filtros; Historial de Eventos (archivado automático + restaurar/duplicar, `/organizador/historial`). Reglas de publicación: organización aprobada, ubicación completa, al menos una función, al menos un tipo de entrada con precio y stock, cada función con al menos una asignación de entradas habilitada. **Limitaciones:** no hay reordenamiento manual de eventos en el listado más allá del orden por fecha de creación.

Funciones

Propósito: programar las fechas/horarios concretos de un evento, independientes del catálogo de entradas. **Estado: completo.** Programación por fecha única, rango de fechas, días de la semana o recurrencia (Web y WhatsApp, mismas tres modalidades en ambos canales); pantalla dedicada "Estado de Funciones" (`/organizador/funciones`) con capacidad/emitidas/vendidas/check-ins por función, integrando también las estadísticas de cortesías; cada función puede tener asignaciones de tipos de entrada con overrides de precio/stock/visibilidad respecto del catálogo general.

Entradas y tipos de tickets

Propósito: catálogo de tipos de entrada (General, VIP, etc.) por evento y administración de las entradas ya vendidas. **Estado: completo.** *Catálogo* (`/organizador/tipos-de-entrada`): alta/edición/baja de tipos de entrada, overrides por función, columna "Vendidas" con datos reales por tipo de entrada (`GET /api/events/mine/ticket-types-sales`). *Administración de entradas vendidas* (`/organizador/entradas`): modelo con auditoría completa (`TicketAuditLog`, `append-`

only) y check-ins históricos (`CheckIn` , ya no 1:1 con la entrada); 5 operaciones administrativas (cancelar/rehabilitar/reactivar/marcar usada manualmente/eliminar con soft delete), individuales o en lote; buscador server-side (número/nombre/email/DNI); selector de evento/función; **filtro por estado con control visual real** (pills clickeables, ya conectadas al backend). **Limitaciones:** la acción masiva "Exportar seleccionadas" está en la interfaz pero es un stub — muestra un aviso ("se preparará en una próxima iteración") sin generar ningún archivo.

Ventas

Propósito: que el organizador vea y gestione las operaciones de venta de sus eventos. **Estado: completo.** Pantalla dedicada (`/organizador/ventas`) con listado filtrable por evento/estado (todas/confirmadas/pendientes/canceladas/vencidas) y búsqueda, alimentada por `GET /api/sales` . Confirmación manual de ventas cargadas a mano (efectivo/transferencia) vía `POST /api/sales/:id/confirm` . El equipo Developer tiene su propia vista platform-wide equivalente, de sólo lectura (`/developer/ventas`).

QR y control de acceso

Propósito: emitir entradas verificables y validarlas en la puerta sin ambigüedad. **Estado: completo.** Al confirmarse una venta, cada `Ticket` recibe un `TicketQr` cuyo secreto se genera con un generador criptográfico seguro y se cifra (AES-256-GCM) antes de guardarse — nunca en texto plano ni como imagen persistida; se arma al vuelo cada vez que hace falta mostrarlo (email, "Mis entradas", PDF). La validación en el Scanner es en **dos pasos:** `scan` (lee el QR, devuelve el estado de la entrada — válida/ya usada/cancelada/no corresponde a este evento — y datos para mostrar al operador: nombre del comprador, función, lugar — sin mutar nada) y `confirm` (recién ahí marca la entrada como usada, con un `UPDATE` condicional atómico dentro de una transacción). Esa separación deja ver quién está entrando antes de confirmar el ingreso. La atomicidad del segundo paso está verificada con pruebas de concurrencia real (hasta 20 escaneos simultáneos sobre la misma entrada, nunca un doble ingreso válido).

Scanners

Propósito: operar el control de acceso en la puerta, sin cuenta de usuario tradicional. **Estado: completo.** Alta de invitaciones por el organizador (una o varias, por puerta); registro público único por invitación (datos personales + código de 6 dígitos); portal de acceso recurrente "Soy Scanner" (email + código, sin contraseña, sin Clerk) para todo ingreso posterior; sesión propia por JWT (24 hs), revalidada `ACTIVE` en cada request; dashboard previo, selección de evento/función, lector de QR con cámara, historial de escaneos, estadísticas de función. El equipo Developer tiene una vista platform-wide de sólo lectura equivalente (`/developer/scanners`). **Limitaciones:** no hay una vista de "todas las puertas en vivo" para el organizador durante el evento (sólo historial/estadísticas por función).

Cortesías

Propósito: que el organizador emita entradas sin costo (sponsors, prensa, invitados, staff, etc.) sin mezclarlas con la venta real. **Estado: completo.** Asistente de 6 pasos (evento → función → tipo de entrada del catálogo existente → cantidad → motivo opcional + nota libre → entrega por "Compartir" o "Enviar por correo"); reutiliza de punta a punta el mismo flujo de venta confirmada

(mismo `Ticket`, QR cifrado, PDF, Scanner) — la única diferencia real es `origin=COURTESY` en vez de `SALE` y precio 0; historial con filtros, estado derivado en vivo, auditoría completa por emisión. Ninguna métrica comercial (recaudación, ticket promedio) incluye nunca una cortesía — todas filtran explícitamente `origin=SALE`. **Limitaciones:** un Developer no puede cancelar una cortesía de una organización ajena; el historial no tiene todavía un filtro visible por canal de emisión (el dato ya existe, falta sólo la UI).

Recuperación de compra

Propósito: que alguien que compró sin cuenta pueda volver a ver/descargar sus entradas. **Estado: completo.** Búsqueda por email+DNI (respuesta siempre genérica, nunca revela si existe o no una coincidencia), código de verificación de 6 dígitos por email como segundo factor, pantalla intermedia "Compra encontrada" antes de exponer cualquier dato, reenvío del email completo, descarga de PDF, reutilización total del flujo de compra (`saleToken`) para "Ver mis entradas".

Emails

Propósito: notificaciones transaccionales. **Estado: completo para lo que existe, acotado en alcance.** Tres emails reales, los tres vía Resend: confirmación de compra (QR embebidos + PDF adjunto, protegido contra duplicados con un reclamo atómico), código de verificación de Scanner (6 dígitos, 10 minutos), código de verificación de recuperación de compra (6 dígitos, 10 minutos, sin mencionar ningún dato de la compra). **Limitaciones:** no hay email de bienvenida, no hay notificación de aprobación/rechazo de organización, no hay notificación de venta nueva al organizador.

Panel Developer

Propósito: panorama y control de toda la plataforma para el equipo de PaseCultural. **Estado: completo, con datos reales de punta a punta.** Dashboard con KPIs reales (usuarios, organizadores, organizaciones pendientes, eventos publicados, entradas vendidas, volumen bruto de ventas, scanners activos), próximos eventos y actividad reciente combinada (altas/aprobaciones de organización, altas/publicaciones de evento, ventas confirmadas) — todo calculado server-side, sin ningún valor hardcodeado. Vistas platform-wide de sólo lectura: organizaciones (aprobar/rechazar/suspender), usuarios (rol/estado/baja), eventos, entradas, scanners, ventas. Panel de Herramientas de Desarrollo (`/developer/base-de-datos`): estadísticas de la base, reset completo (con frase de confirmación exacta del lado servidor, no sólo del lado cliente), creación de evento de demostración, envío de un mensaje de WhatsApp de prueba. **Limitaciones:** la protección del panel de Herramientas de Desarrollo es deliberadamente simple (sólo `requireRole("DEVELOPER")` + frase de confirmación para el reset) — el propio código la documenta como "temporal", asumiendo que todavía no hay clientes reales en producción; esa premisa **no pudo verificarse de forma independiente** desde este entorno (implicaría consultar producción, fuera de alcance).

Panel Organizer

Propósito: panorama del organizador sobre su propia organización. **Estado: completo, con datos reales de punta a punta.** Banner de estado de la organización; selector de evento destacado; hero con ocupación real; resumen del evento en 4 bloques — **Comercial** (recaudación,

entradas vendidas, ticket promedio, sólo `origin=SALE`), **Emisión** (emitidas totales + desglose por canal), **Accesos** (ingresadas/pendientes/canceladas) y **Ocupación** (capacidad/emitidas/disponibles/%) — todos calculados 100% en el backend (`functionCapacity.service.js`), única fuente compartida con la pantalla de Entradas; actividad reciente; grilla "Estado de mis eventos" (capacidad/ocupación reales, endpoint `batcheado`); últimas ventas. **Limitaciones:** la "recaudación" mostrada es la suma de ventas ya `CONFIRMED` en la base — no hay cobro real detrás (ver Mercado Pago).

Configuración

Propósito: que el organizador administre los datos de su organización. **Estado: completo salvo cobros.** Edición de todos los datos de la organización (nombre, tipo, CUIT, contacto, ubicación, redes, logo, descripción); vinculación de un número de WhatsApp a la organización, con verificación por código de 6 dígitos enviado al propio WhatsApp (`GET/POST /api/organizations/me/whatsapp-link`) — mismo mecanismo de vínculo que usa el descubrimiento automático por teléfono desde el lado del bot (ver §5), acá iniciado proactivamente desde la Web. **Limitaciones:** la tarjeta "Datos bancarios / Mercado Pago" es un placeholder estático, sin ningún campo ni lógica.

WhatsApp

Propósito: canal alternativo de creación de eventos, conversando con un bot desde WhatsApp. **Estado: completo y en uso,** mismo motor que la Web. Detalle completo del flujo vigente en §5.

Mercado Pago

Propósito: cobrar de verdad y confirmar la venta según el resultado del pago. **Estado: NO implementado.** Existe el valor `MERCADO_PAGO` en el enum `PaymentMethod` (nunca usado en la práctica — todas las ventas se crean con `MANUAL`), un campo `Sale.confirmedBy` nullable (preparado para que una confirmación automática por webhook no tenga un organizador asociado), y un único punto de extensión aislado en el frontend (`processPayment()` , documentado como "la única función que va a cambiar el día que se integre Mercado Pago"). Hoy esa función crea la venta y la confirma en el mismo paso, sin pasarela ni redirección a ningún checkout externo. Confirmación manual por organizador sí existe (pensada para efectivo/transferencia). **Falta todo:** SDK, checkout, webhook de notificación, conciliación de estados, manejo de pagos rechazados/pendientes, onboarding de cuenta de cobro.

5. FLUJO ACTUAL DE WHATSAPP

Todo lo que sigue describe el comportamiento **vigente** del bot conversacional para organizadores — no su historia. El motor real que impulsa cada paso es el mismo `EventCreationEngine / EventServicePort` que usa la Web; WhatsApp sólo aporta la capa de interpretación de texto libre y los mensajes propios del canal.

Saludo y descubrimiento de organizaciones. Un mensaje entrante se identifica por el teléfono de origen (`wa_id`). Si ese teléfono ya tiene una organización vinculada (`WhatsappOrganizerLink`), se resuelve directo. Si no, se buscan en paralelo (`Promise.all`, nunca secuencial) los vínculos ya existentes y las organizaciones aprobadas cuyo teléfono coincide, y se combinan deduplicando exclusivamente por `organizationId` — así una organización con el mismo teléfono pero sin vínculo creado todavía también se descubre.

Selección cuando el teléfono pertenece a varias organizaciones. Siempre se pregunta explícitamente "¿Con cuál de tus organizaciones querés trabajar?", listando todas las candidatas — nunca se asume una por defecto.

Vinculación de un teléfono nuevo. Si el número no está vinculado a ninguna organización conocida, se ofrece un código de verificación de 6 dígitos (vigente 10 minutos, máximo 5 intentos, cooldown de reenvío de 1 minuto) — mismo mecanismo que la vinculación proactiva desde Configuración en la Web (§4).

Creación y reanudación de conversaciones. Cada conversación es una fila `ConversationState` (`channel="WHATSAPP"`, `channelRef=wa_id`). Al llegar un mensaje, se busca la conversación `ACTIVE` de ese `wa_id` (única consulta adicional que necesita WhatsApp y que Web no necesita, porque Web siempre trae su `conversationId` guardado del lado del cliente); si existe, se retoma exactamente donde quedó — el organizador puede cortar y volver horas después sin perder nada.

Carga paso a paso del evento. El mismo `draftEvent` en JSON que arma el motor para Web se va completando pregunta por pregunta; nada se persiste como `Event` real hasta que el organizador confirma "Publicar" o "Guardar borrador" en la vista previa final.

Modalidades de funciones. Las mismas tres que en Web: una sola función, varias funciones cargadas una por una, o un rango recurrente por días de la semana.

Carga de imágenes. El organizador manda una foto real → se descarga desde Meta y se sube a Cloudinary → el motor avanza. Si en el paso que espera la imagen llega **cualquier otra cosa** — texto, video, audio, documento, sticker o una ubicación — el bot responde siempre con el mismo mensaje claro y específico de WhatsApp ("Necesito que envíes una foto del evento... Usá el botón de adjuntar..."), nunca con el error genérico pensado para el flujo Web (que menciona un endpoint HTTP). El estado no se corrompe ni avanza en ningún caso: el `draftEvent` /paso actual quedan exactamente donde estaban hasta que llega una imagen real.

Ubicación: compartida o dirección manual. Al llegar al paso de ubicación, el bot pregunta directo "¿Cómo querés cargar la ubicación? 1. Compartir ubicación / 2. Completar dirección manualmente" — **el sistema no sugiere ni precarga ninguna dirección de un evento anterior de la organización.** El estado arranca siempre vacío (sin dirección, coordenadas ni ningún dato precargado). Compartir ubicación nativa entrega coordenadas reales; la dirección

manual se carga en un solo mensaje separado por comas (calle y altura, ciudad, provincia). En ambos casos se muestra un resumen de confirmación (con link a Google Maps) **después** de que el organizador ingresó la dirección o compartió la ubicación — nunca antes. El status `AWAITING_REUSE_CONFIRMATION` existe todavía en el código exclusivamente por compatibilidad retroactiva: si una conversación vieja quedó esperando esa respuesta (de antes de este cambio), el bot todavía sabe procesarla para que no quede trabada, pero ningún mensaje nuevo la genera.

Creación DRAFT. Al elegir "Guardar borrador" en la vista previa, se crea el `Event` real (`status="DRAFT"`), sus funciones, catálogo de entradas y links — sin publicarlo. La lectura final que arma la respuesta usa un camino optimizado (ver abajo) que no vuelve a resolver la organización ni corre el self-heal de archivado (matemáticamente imposible que encuentre algo en un evento recién creado).

Publicación. Al elegir "Publicar", además de lo anterior se valida que la organización esté aprobada y que el evento cumpla las reglas de publicación (§4, Eventos), y el evento pasa a `PUBLISHED` . Esta operación es la más lenta del flujo (~20 segundos) — aceptada como está por ser una operación única por evento, no repetida por cada mensaje (ver abajo).

Cancelación y navegación. "Cancelar" borra por completo el borrador en curso. "Volver" retrocede un paso sin perder lo ya cargado más adelante (nunca revierte el `draftEvent` , sólo mueve el cursor). Cualquier paso puede editarse puntualmente desde la vista previa final antes de confirmar.

Comportamiento ante errores. Un rechazo de validación del motor (ej. falta la ubicación, categoría personalizada sin especificar) nunca corta la conversación — vuelve como un mensaje de error conversacional sobre el mismo paso, con el estado intacto.

Diseño de rendimiento del canal

Estas son características actuales del diseño, no un historial de cambios:

- **Cache por mensaje de `ConversationState`** . Cada mensaje entrante corre dentro de su propio contexto aislado (`AsyncLocalStorage`) — nunca compartido entre mensajes concurrentes de distintos números, nunca reutilizado entre un mensaje y el siguiente del mismo número.
- **Camino habitual en 2 consultas.** Encontrar la conversación activa y aplicar la escritura de un paso normal son `findFirst` + `updateMany` — la fila completa que trae ese `findFirst` queda cacheada en el contexto del mensaje, así que no hace falta un segundo `findUnique` para retomarla.
- **Detección de conflictos con `status: ACTIVE`** . Toda escritura filtra explícitamente por `status: "ACTIVE"` en vez de un `update` ciego por id — si otro proceso cerró/eliminó la conversación justo en el medio (siempre detectable, nunca silenciado), se invalida la copia cacheada y se lanza el error real correspondiente en vez de escribir sobre una fila que ya no correspondía.
- **`WhatsappPendingStepInput.upsert`** . Los sub-pasos intermedios (método de ubicación, dirección compuesta, etc.) se guardan con una única operación `upsert` , nunca un borrado seguido de una creación.
- **Descubrimiento de organizaciones paralelizado.** Las dos consultas independientes de la resolución por teléfono corren con `Promise.all` , nunca una detrás de la otra.

- **Lectura final de DRAFT optimizada.** No repite la resolución de usuario/organización que ya se hizo al crear el evento en la misma llamada, y se salta un self-heal de archivado que no puede encontrar nada en un evento recién creado.
 - **PUBLISH sin cambios.** La rama de publicación no fue tocada por ninguna de estas optimizaciones — mismo comportamiento y mismo costo de siempre, los ~20 segundos mencionados arriba se aceptan como una latencia conocida de una operación que ocurre una única vez por evento.
-

6. BACKEND Y API

Organización de carpetas

`backend/src/{controllers,services,routes,middlewares,utils,config,conversation,errors,logging}`. Un archivo por dominio en `controllers/ / routes/`, patrón consistente: el controller sólo valida `req`, llama al service y devuelve la respuesta; la lógica de negocio vive en los services (`whatsapp.controller.js` es la única excepción parcial — por diseño concentra los sub-flujos de interpretación de texto libre de ese canal).

Servicios clave y su responsabilidad

- `event.service.js` / `functionCapacity.service.js` — CRUD de eventos y única fuente de capacidad/emitidas/vendidas/ingresadas/canceladas (por función, por evento y batcheada).
- `sale.service.js` / `courtesy.service.js` — ventas reales y cortesías, esta última reutilizando la primera en vez de duplicar lógica.
- `ticketAdmin.service.js` — las 5 operaciones administrativas sobre entradas, individuales y masivas, cada una auditada.
- `scanner.service.js` / `scannerLogin.service.js` / `scannerInvitation.service.js` / `eventScanner.service.js` — todo el ciclo de vida del módulo Scanner.
- `developerDashboard.service.js` / `developerEvents.service.js` / `developerSales.service.js` / `developerScanners.service.js` / `developerTickets.service.js` — vistas platform-wide de sólo lectura para Developer, siempre sin `organizationId` como filtro (a propósito).
- `devTools.service.js` — herramientas de desarrollo (reset/seed), protección deliberadamente simplificada (ver §4/§8).
- `conversation/EventCreationEngine.js` + `EventServicePort.js` — el motor conversacional compartido por Web y WhatsApp.
- `whatsapp*.service.js` (7 archivos) — todo lo específico del canal: envío/parseo Meta, descubrimiento por teléfono, vínculo organizador↔teléfono, pending steps, ubicación reutilizable (ya no llamada desde el flujo activo, ver §5), subida de imágenes.
- `email/` — los 3 servicios de envío transaccional (ver §4, Emails) + generación de QR/PDF embebidos.
- `eventArchive.service.js` — self-heal de archivado automático, reusado por todos los listados operativos.

Manejo de errores

`errors/AppError.js` centraliza el mapeo de errores de negocio a respuestas HTTP; `errors/errorHandler.js` es el único middleware de error de la app (`app.js`). El motor conversacional traduce errores internos a mensajes conversacionales (`translateEventServiceError`) para no filtrar nunca un error técnico crudo al organizador.

Integraciones externas

Ver §2 (Arquitectura) para la lista completa; cada integración vive detrás de un único módulo de configuración en `src/config/` (`prisma.js`, `resend.js`, `cloudinary.js`, `qrEncryption.js`, `scannerSession.js`) o de un service dedicado (`whatsapp.service.js`, `geocoding.service.js`).

Inventario de endpoints (agrupado por dominio)

Públicos: `GET /api/health` · `GET /api/events/public`, `/public/:slug`, `/categories` · `POST /api/sales`, `POST /api/sales/:token/confirm-by-buyer`, `GET /api/sales/:token/status`, `GET /api/sales/:token/pdf`, `POST /api/sales/:token/resend-email` · `POST /api/sales/recover`, `/recover/resend`, `/recover/verify` · `GET /api/scanner-invitations/:token`, `POST /:token/register`, `/:token/resend`, `/:token/verify` · `POST /api/scanner-auth/request-code`, `/resend-code`, `/verify` · `GET/POST /api/whatsapp/webhook`.

Auth: `POST /api/auth/sync`.

Usuarios (Developer): `GET /api/users`, `/count`, `/:id` · `PATCH /:id/role`, `/:id/status` · `DELETE /:id`.

Organizaciones: `GET/PATCH/DELETE /api/organizations/me` · `POST /api/organizations` · `GET/POST /api/organizations/me/whatsapp-link` (Organizer) · `GET /api/organizations`, `/:id` · `PATCH /:id/status` · `DELETE /:id` (Developer).

Eventos: `GET /api/events/scanner-events`, `/archived` · `POST /api/events` · `GET /mine`, `/mine/stats`, `/mine/ticket-types-sales`, `/:id` · `PATCH /:id` · `PUT /:id/schedule`, `/:id/links` · `POST /:id/restore`, `/:id/duplicate` · `GET /:id/functions/stats` · `DELETE /:id`. Sub-recursos de scanners y administración de entradas de un evento anidados bajo `/api/events/:id/...` (8 + 5 endpoints respectivamente, altas/bajas/acciones individuales y masivas).

Conversación (chat de creación de eventos, Web): `POST /api/conversations/start` · `GET /:id/status`, `/:id` · `POST /:id/reply` · `DELETE /:id`.

Ventas: `GET /api/sales/mine` · `GET /api/sales` (Organizer) · `POST /:id/confirm` (Organizer) · `POST /:id/cancel` · `POST /:id/resend-confirmation-email` (Developer/Organizer).

Cortesías (Organizer/Developer): `POST /api/courtesies` · `GET /api/courtesies`, `/stats` · `POST /:saleId/resend-email` · `GET /:saleId/pdf` · `POST /:saleId/cancel`.

Entradas (comprador): `GET /api/tickets/mine`, `/number/:ticketNumber`, `/:id/qr`, `/:id`.

Entradas (organizador): `GET /api/tickets/organizer`.

Medios: `POST /api/media/upload` · `DELETE /api/media/*publicId`.

Módulo Scanner (sesión propia): `GET /api/scanner/dashboard`, `/events`, `/events/:eventId/functions/:functionId/stats`, `/scan-attempts` · `POST /scan`, `/scan/confirm`.

Panel Developer (todos `requireRole("DEVELOPER")`): `GET /api/developer/dashboard` · `GET /events`, `/organizations/options` · `GET /sales`, `/sales/:id` · `GET /scanners`, `/scanners/:id` · `GET /tickets`, `/tickets/:id`, `/events/options`.

Herramientas de Desarrollo (`/api/dev`, Developer): `GET /stats` · `POST /reset`, `/demo-event`, `/whatsapp/test-send`.

7. BASE DE DATOS

schema.prisma: **22 modelos**, **22 enums**, 34 migraciones aplicadas. No se incluye ninguna credencial ni URL de conexión en este documento — ver §8 para cómo se referencian ambos entornos (producción/TEST) de forma segura, siempre por project-ref, nunca por URL completa.

Modelos principales

- **User** — cuenta de cualquier persona; `clerkId` nullable (compra de invitado). Relaciona con `Organization` (dueño), `Sale / Ticket` (comprador).
- **Organization** — entidad organizadora, dueña de sus eventos, con ciclo de aprobación por Developer.
- **Event** — evento cultural; entidad central del catálogo. Relaciona con `EventFunction`, `TicketType`, `EventLink`, `Sale`, `Ticket`, `EventScanner`.
- **EventFunction** — una fecha/función concreta (precio/stock viven en `TicketType`, no acá).
- **TicketType** — catálogo reusable de tipos de entrada por evento.
- **FunctionTicketType** — asigna un `TicketType` a una `EventFunction`, con overrides opcionales.
- **EventLink** — link asociado a un evento, con metadata de embed resuelta server-side.
- **Sale** — una venta/orden. `origin` (`SaleOrigin`: `SALE / COURTESY`) distingue venta real de cortesía sin duplicar el modelo; toda métrica comercial filtra explícitamente `origin=SALE`.
- **SaleItem** — línea de detalle de una venta (tipo de entrada + cantidad + precio congelado).
- **CourtesyIssuance** — metadata de una cortesía (motivo, nota, método de entrega), 1 a 1 con la `Sale` que representa.
- **Ticket** — entrada individual escaneable. Tiene su propio `origin` (denormalizado desde `Sale.origin`, necesario porque Prisma no puede agrupar por un campo de una relación) — es lo que permite desglosar emitidas/vendidas por canal sin join costoso, y admite cualquier canal futuro agregando sólo un valor al enum.
- **TicketQr** — secreto del QR, cifrado de forma reversible, nunca en texto plano ni como imagen.
- **CheckIn** — historial de ingresos (ya no 1:1 con `Ticket` — una entrada reactivada puede volver a tener check-ins).
- **ScanAttempt** — auditoría de TODO intento de escaneo, válido o no.
- **TicketAuditLog** — bitácora append-only de toda acción administrativa sobre una entrada.
- **EventScanner** — identidad completa de un puesto de scanner (invitación + verificación), sin cuenta de Clerk.
- **SaleRecoveryVerification** — sesión de código de 6 dígitos para recuperación de compra, clave única por email+DNI normalizado.

Tablas del flujo WhatsApp

- **ConversationState** — estado persistido del motor conversacional (borrador en curso, paso actual, historial de navegación); compartido por Web y WhatsApp, distinguidos por `channel / channelRef`.

- **WhatsappOrganizerLink** — vínculo `wa_id` ↔ `Organization` (`organizationId` único: una `Organization`, a lo sumo un vínculo; `waId` deliberadamente no único, un teléfono puede administrar más de una organización).
- **WhatsappLinkChallenge** — código de 6 dígitos pendiente de verificación para crear un vínculo nuevo.
- **WhatsappPendingOrganizationSelection** — estado del selector cuando un teléfono tiene más de una organización candidata.
- **WhatsappPendingStepInput** — sub-pasos intermedios de un paso compuesto (ej. método de ubicación → dirección), con `conversationId` único (a lo sumo un pending por conversación).

Enums

`Role`, `UserStatus`, `OrganizationStatus`, `OrganizationType`, `EventStatus`, `EventVisibility`, `FunctionStatus`, `ConversationChannel`, `ConversationStatus`, `SaleStatus`, `SaleOrigin`, `PaymentMethod`, `EmailDeliveryStatus`, `TicketStatus`, `ScanResult`, `CourtesyReason`, `CourtesyDeliveryMethod`, `CheckInSource`, `TicketAuditAction`, `TicketAuditActorType`, `EventScannerStatus`.

Datos sensibles

El secreto de cada QR se cifra en la base (AES-256-GCM, clave fuera de la base). Ningún log de la aplicación imprime teléfono completo, DNI, email o el secreto de un QR — los logs de WhatsApp (`[WA_PERF]`) truncan el `conversationId` a 8 caracteres y nunca incluyen texto de mensajes ni identificadores de Meta.

8. SEGURIDAD Y CONSISTENCIA

Clerk y autorización. Dos sistemas de autenticación deliberadamente no unificados: Clerk para cualquier cuenta con sesión (comprador, organizador, developer); JWT propio, sin Clerk, para el módulo Scanner. Autorización siempre por rol server-side (`requireRole`) más, dentro del panel de organizador, verificación de pertenencia del recurso a la organización de quien pide — nunca alcanza con el rol solo.

Cifrado de QR. Generador criptográfico seguro + cifrado reversible (AES-256-GCM, clave en variable de entorno, nunca en la base) + comparación en tiempo constante al validar un escaneo (protección contra timing attacks).

Control de ownership. Cada acción sobre un evento, scanner o entrada revalida explícitamente que ese recurso pertenece a la organización de quien la pide — verificado en cada service, no sólo en el middleware de rol.

Transacciones y actualizaciones atómicas. El paso `confirm` de un escaneo usa un `UPDATE` condicional (`status: "ACTIVE" → "USED"`) dentro de una transacción — verificado con pruebas de concurrencia real, hasta 20 escaneos simultáneos sobre la misma entrada, sin nunca un doble ingreso válido. `ConversationState` usa el mismo patrón (`updateMany` filtrando por `status: "ACTIVE"` en vez de un `update` ciego) para detectar si otra escritura tocó la fila en el medio. Operaciones de escritura múltiple (sincronizar funciones+catálogo de un evento, reset completo de base) corren dentro de `$transaction`.

Guardrail `dbGuard.js`. Módulo centralizado que lee `DATABASE_URL / DIRECT_URL` y aborta de inmediato — antes de que cualquier test defina un solo caso, mucho antes de poder crear un fixture — si detecta el project-ref de **producción**; habilita los tests con base real (`hasDatabase = true`) sólo si detecta el project-ref de **TEST**; nunca imprime la connection string completa, sólo compara como substring contra los dos project-ref conocidos. Está **commiteado y pusheado a main**, activo en el repositorio actual. Protege actualmente **4 archivos** de test que tocan Prisma real (no 3): `whatsappOrganizerDiscovery.test.js`, `whatsappPendingStepInput.service.test.js`, `eventServicePort.commit.perf.test.js`, `eventCreationEngine.conversationStateCache.test.js`.

Separación TEST/producción. `backend/.env.test` **existe** en este entorno (no versionado en git, cargado sólo por `npm run test:db / npm test` vía un loader dedicado que nunca lee `.env`) y apunta al project-ref de TEST. `npm run test:db` **ya se ejecutó exitosamente contra la base de TEST real**, en más de una ocasión, siempre después de confirmar el project-ref con `dbGuard.js` antes de correr — ver §9 para los últimos resultados verificados.

Protección de concurrencia — alcance real. WhatsApp **detecta** conflictos de concurrencia sobre una misma conversación (la escritura filtrada por `status: "ACTIVE"` nunca pisa una fila que cambió en el medio, y lanza el error correcto en vez de fallar en silencio) — pero **todavía no serializa** todos los mensajes entrantes de un mismo `wa_id`: dos mensajes casi simultáneos del mismo número no se encolan ni se bloquean entre sí, sólo se detecta el conflicto si efectivamente llegan a pisarse. En el uso real (un organizador difícilmente manda dos mensajes en el mismo milisegundo) esto no se ha observado como problema, pero es una garantía distinta — más débil — que la de bloqueo por fila real que sí tiene el módulo Scanner.

Deduplicación de reintentos de Meta. No hay ningún mecanismo de deduplicación por `wamid / messageId` confirmado en el código — Meta puede reintregar el mismo webhook más de una vez ante una demora en la respuesta; no está implementado ningún chequeo que reconozca "este mensaje ya se procesó" antes de reintentarlo como si fuera nuevo.

Rate limits. Limitador propio en memoria (sin dependencia externa, no apto para múltiples instancias sin adaptarlo), aplicado a todos los endpoints públicos "adivinables" (recuperación de compra, invitación/login de Scanner).

Panel de Herramientas de Desarrollo. Ver §4 (Panel Developer) — protección deliberadamente simplificada, documentada como temporal en el propio código.

9. INSTALACIÓN, PRUEBAS Y DESPLIEGUE

Requisitos

Node.js (probado con v24.14.0 en este entorno; el proyecto no declara una versión mínima en `package.json`), npm, acceso a un proyecto PostgreSQL de Supabase (producción para correr la app; uno separado de TEST sólo para `npm run test:db`).

Variables de entorno (nombres únicamente — nunca valores)

Backend (`backend/.env`, no versionado): `PORT`, `NODE_ENV`, `DATABASE_URL`, `DIRECT_URL`, `CLERK_SECRET_KEY` (leída por `@clerk/express` por convención del SDK), `SCANNER_SESSION_SECRET`, `TICKET_QR_SECRET_KEY`, `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`, `RESEND_API_KEY`, `EMAIL_FROM`, `EMAIL_REPLY_TO` (opcional), `FRONTEND_URL`, `GOOGLE_MAPS_SERVER_API_KEY`, `WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_VERIFY_TOKEN`, `WHATSAPP_GRAPH_API_VERSION`, `WHATSAPP_TEST_MODE` (opcional), `WHATSAPP_PERF_LOG` (opcional, diagnóstico). `RENDER_GIT_COMMIT` la inyecta Render automáticamente, nunca se configura a mano.

Backend, sólo tests con DB (`backend/.env.test`, no versionado — plantilla en `backend/.env.test.example`): `DATABASE_URL`, `DIRECT_URL` — deben apuntar exclusivamente al project-ref de TEST.

Frontend (`frontend/.env`, no versionado, prefijo `VITE_` requerido por Vite): `VITE_API_URL`, `VITE_CLERK_PUBLISHABLE_KEY`, `VITE_GOOGLE_MAPS_API_KEY`.

Cómo levantar el proyecto localmente

```
cd backend && npm install && npm run dev      # nodemon, puerto según PORT
cd frontend && npm install && npm run dev      # Vite dev server
```

`npm run build` (frontend) genera el bundle estático que Vercel sirve.

Comandos de test y su diferencia

- `npm test` — suite completa, carga `.env.test` (vía `--import ./tests/helpers/loadTestEnv.js`) antes de que cualquier test se registre.
- `npm run test:unit` — **no** carga `.env.test`; cualquier test que dependa de una base real (`hasDatabase`) se salta automáticamente. Nunca toca una base de datos.
- `npm run test:db` — corre exclusivamente los 4 archivos que tocan Prisma real (ver §8, `dbGuard`), con `.env.test` cargado.

Guardrail antes de usar DB

Antes de correr `npm test` / `npm run test:db`, `dbGuard.js` ya valida automáticamente el project-ref al importarse — pero como verificación manual adicional recomendada: confirmar que `backend/.env.test` contiene el project-ref de TEST y no el de producción, y que no hay ninguna

variable `DATABASE_URL / DIRECT_URL` exportada en la shell que pueda pisar ese archivo (`dotenv` no sobrescribe variables ya presentes en el entorno).

Despliegue

Backend: Render, configurado vía su propio dashboard (sin `render.yaml` versionado en este repositorio) — deploy típico a partir de push a `main` . **Frontend:** Vercel, integración estándar Git — `frontend/vercel.json` sólo define la regla de rewrite SPA.

Últimos resultados verificados

Comando	Total	Pass	Fail	Skipped	Cuándo / contra qué
<code>npm run test:unit</code>	556	505	0	51	Verificado en este entorno, 2026-08-14 — sin base de datos, los 51 omitidos son exclusivamente los que requieren DB real
<code>npm run test:db</code>	66	66	0	0	Verificado en este entorno, 2026-08-14, contra el project-ref de TEST, después de aplicar todos los cambios de código de la fecha

Pruebas automatizadas vs. pruebas manuales

Los resultados de arriba son de la suite automatizada (`node:test`), ejecutada en este entorno. Las verificaciones de comportamiento descritas en los módulos de este informe (ej. §5, manejo de imágenes/ubicación en WhatsApp) están respaldadas por lectura directa y verificada del código fuente vigente — no por una corrida en vivo del bot contra un número de WhatsApp real. **No se ejecutó, como parte de la elaboración de este documento, una prueba manual continua de punta a punta del flujo completo de WhatsApp** (saludo → imagen → ubicación → funciones → entradas → vista previa → publicación) en un único recorrido — ver §10 para lo que queda pendiente de esa validación funcional.

10. LIMITACIONES Y PENDIENTES REALES

Sólo lo que sigue efectivamente pendiente, verificado contra el código vigente — nada de esto está ya resuelto:

1. **Integración de Mercado Pago.** Es la ausencia más grande del sistema — sin esto, ninguna venta cobra dinero de verdad. Todo el resto (entradas, QR, scanner, emails) ya está construido asumiendo que este paso va a existir.
 2. **Deduplicación de reintentos de Meta por `wamid`.** No implementada — ver §8.
 3. **Serialización de mensajes de WhatsApp por `wa_id`.** Hoy sólo hay detección de conflicto, no bloqueo/cola — considerar si aparece evidencia real de mensajes casi simultáneos del mismo número causando algún problema.
 4. **Timeout de conversación de WhatsApp por inactividad.** No hay campo de expiración en `ConversationState` ni ningún job/cron en el proyecto (no existe infraestructura de tareas programadas) que pudiera aplicarlo — una conversación abandonada queda `ACTIVE` indefinidamente hasta que el organizador la retoma o la cancela.
 5. **Revalidación funcional de fechas anteriores al día actual.** La regla está implementada en código (`isArgentineDateInThePast` , aplicada en los 3 caminos de carga de fecha) y verificada por lectura de código — falta una revalidación funcional en vivo como parte de una prueba final integral.
 6. **Revalidación funcional de las 3 modalidades de función (única/múltiples/recurrentes).** Mismo caso que el punto anterior: implementado y verificado por código, falta una pasada funcional en vivo.
 7. **Prueba final integral del canal WhatsApp de punta a punta.** No ejecutada como parte de este documento (ver §9).
 8. **Exportación real de entradas seleccionadas.** El botón existe en la UI, la lógica no.
 9. **Página de Términos de Servicio dedicada.** No se encontró una ruta propia en el frontend — sí existen ya `/privacidad` y `/eliminacion-de-datos` como páginas reales.
 10. **Limpieza de datos de prueba en producción.** Un lote de organizaciones de prueba (creadas por una corrida real de tests contra producción antes de que existiera `dbGuard.js`) quedó identificado pero no se confirmó su limpieza desde este entorno — no verificable sin consultar producción, fuera de alcance.
 11. **Recaudación mostrada como cobro real.** Sigue siendo la suma de ventas `CONFIRMED` en la base, no un cobro real — depende enteramente de Mercado Pago.
 12. **Protección temporal del panel de Herramientas de Desarrollo.** Documentada en el propio código como provisoria mientras no haya clientes reales — esa premisa no se pudo reverificar de forma independiente.
-

11. GUÍA DE TRASPASO

Archivos que un desarrollador nuevo debería leer primero

1. Este documento completo.
2. `backend/prisma/schema.prisma` — el modelo de datos real, fuente de verdad de toda relación.
3. `backend/src/app.js` — todos los montajes de rutas y middlewares globales, en un solo lugar.
4. `backend/src/conversation/EventCreationEngine.js` + `EventServicePort.js` — el motor compartido por Web y WhatsApp; entender esto antes de tocar cualquiera de los dos canales.
5. `backend/src/controllers/whatsapp.controller.js` — el controller más grande del proyecto; concentra toda la interpretación de texto libre del bot.
6. `frontend/src/App.jsx` — el árbol completo de rutas y guards por rol.

Cómo agregar una funcionalidad sin romper Web o WhatsApp

Cualquier cambio a un paso del motor conversacional (`conversation/steps/`, `inputHandlers/`) afecta a **ambos** canales — no hay fork de lógica de negocio entre ellos. Si el cambio es específico de un canal (un texto, un formato de mensaje, una validación de formato de entrada), debe vivir en la capa de adaptador de ese canal (`whatsapp.controller.js` / `whatsappOrganizerBot.service.js` para WhatsApp; los componentes React del wizard/chat para Web), nunca en el motor. Antes de cambiar un paso compartido, revisar ambos adaptadores para confirmar que ninguno asume un detalle del formato anterior.

Zonas que requieren especial cuidado

- **ConversationState y su cache por mensaje** (`conversationStateRequestCache.js`) — la cache es exclusivamente por `AsyncLocalStorage`, nunca debe convertirse en una Map global ni persistir entre mensajes distintos.
- **Escrituras a ConversationState** — siempre vía `updateMany` filtrado por `status: "ACTIVE"`, nunca un `update` ciego por id (ver §8).
- **El secreto de cada QR** (`qrEncryption.js`) — nunca debe persistirse ni loguearse en texto plano.
- **El paso confirm de un escaneo** — la atomicidad de esa única transacción es lo que impide un doble ingreso; cualquier refactor debe preservar el `UPDATE` condicional exacto.
- **dbGuard.js** — ningún test nuevo que toque Prisma real debe implementar su propio chequeo de entorno; siempre importar `hasDatabase` desde ahí.
- **Panel de Herramientas de Desarrollo** — su protección es deliberadamente mínima; no asumir que es seguro exponerlo o replicarlo sin agregar una capa extra antes de que haya clientes reales.

Checklist antes de modificar DB, WhatsApp, ventas o QR

- ☐ ¿El cambio toca `schema.prisma` ? → generar una migración real, nunca editar una ya aplicada.

- ☐ ¿El cambio toca un paso del motor conversacional? → verificar el impacto en Web y WhatsApp antes de mergear.
 - ☐ ¿El cambio agrega un test que usa Prisma real? → importar `hasDatabase` de `dbGuard.js`, nunca un chequeo propio de `process.env.DATABASE_URL`.
 - ☐ ¿Vas a correr `npm run test:db`? → confirmar el project-ref de `backend/.env.test` **antes** de ejecutar, nunca asumir.
 - ☐ ¿El cambio toca la emisión o validación de un `TicketQr`? → nunca loguear ni devolver el secreto en texto plano fuera de los puntos ya existentes (email, "Mis entradas", PDF).
 - ☐ ¿El cambio toca `Sale / Ticket.origin`? → confirmar que ninguna métrica comercial nueva deja de filtrar `origin=SALE`.
-

12. HISTORIAL RESUMIDO

Sólo hitos, sin narrar commits ni fases completas.

Fecha	Cambio	Estado
2026-08-08	Redacción de la primera versión de este informe	Reemplazada por esta versión
2026-08-09	Módulo de Ventas del organizador	Implementado
2026-08-09	Páginas de Política de Privacidad y Eliminación de Datos	Implementado
2026-08-09/11	Adaptador de WhatsApp para creación de eventos (motor compartido con Web)	Implementado
2026-08-12	Corrección de descubrimiento de organizaciones por teléfono	Resuelto
2026-08-13	Guardrail de tests contra producción (<code>dbGuard.js</code>)	Implementado, en <code>main</code>
2026-08-13	Primera optimización de latencia de WhatsApp (cache por mensaje, upsert de pending, discovery paralelo)	Implementado
2026-08-14	Optimización de <code>HANDLE_INPUT</code> (2 consultas en el camino habitual)	Implementado
2026-08-14	Optimización de la lectura final de borradores (<code>PREVIEW_PUBLISH</code> sin cambios)	Implementado
2026-08-14	Eliminación de la sugerencia de dirección de eventos anteriores en WhatsApp	Implementado
2026-08-14	Reescritura de este informe como documentación de traspaso	Vigente